

Data Compression

Srujan Gangoor

Dept. of Computer Science, Rutgers University, Camden

56:198:694 - Independent Study

Dr. Sunil Shende

December 19, 2025

1) Introduction

My choice of this field for an independent study was, in part, inspired by a post [1] on Discord’s engineering blog, a platform that I use on a daily basis for the past 5 years. Part of the reason why their savings on web traffic bandwidth costs just from switching from zlib to zstd surprised me is that I knew little about how data compression techniques worked, despite their ubiquitous use in archiving, software delivery, version control systems, and many others.

Modern lossless compression algorithms such as zstd [2] and lz4 [3] are flexible and offer multiple tradeoffs between speed of compression/decompression and compression ratios. They all have their roots in information theory.

2) Information Content and Entropy

Compression problems are often thought of as guessing games [4, p. 70]. This analogy sees the compressor (or the decompressor) as a participant in a guessing game asking binary yes/no questions to guess the next symbol in a source. This naturally lends to the notion of encoding the answer as a bit that can take on the values 1 (for the answer yes) or 0 (conversely, no). An optimal compression algorithm can then be thought of as “what is the least number of questions one can ask to correctly guess the next symbol?”.

In more general terms, this is defined as the Shannon information content of a random variable x taking on an outcome a_i . X is a triple (x, A_X, P_X) where the outcome x is the value of a random variable that can take on a set of values $A_X = (a_1, a_2, \dots, a_{n-1})$, each of them having a probability from $P_X = (p_1, p_2, \dots, p_{n-1})$, the Shannon information content of an outcome is given by [4, p. 68],

$$h(x = a_i) = \log_2 \left(\frac{1}{p_i} \right) \quad (1)$$

Additionally, the average Shannon information content of the entire ensemble is given by the entropy of X ,

$$H(X) = \sum_i p_i \log_2 \left(\frac{1}{p_i} \right) \quad (2)$$

The information content of variables can help shed light on the entire source given its probability distribution. Not every source has a uniform distribution — the English alphabet has an uneven distribution with ‘e’ being the most used letter in texts and dictionaries. The information content for the

random variable taking on the letter ‘e’ would be quite low, as we cannot guess the next character with a higher degree of certainty than if the word started with a rare letter such as ‘z’.

Lossless compressors can only compress data so much without losing information. This is because while such compressors can shorten some string of symbols, it **must** also lengthen some others. An optimally designed compressor is one that reduces the probability of a source string being lengthened after compression.

2.a) Typicality

A typical string is defined as one with N symbols, each with probability p_i . It contains $p_i N$ occurrences of each symbol, given the probability of the typical string as follows [4, pp. 79-80]:

$$P(x)_{\text{typ}} = P(x_1)P(x_2)...P(x_N) \simeq p_1^{(p_1 N)} p_2^{(p_2 N)} ... p_I^{(p_I N)} \quad (3)$$

which leads to the information content of a typical string:

$$\log_2 \left(\frac{1}{P(X)} \right) \simeq N \sum_i p_i \log_2 \left(\frac{1}{p_i} \right) = NH \quad (4)$$

A typical set, defined using the above observation, is one which contains elements whose probabilities are close to 2^{-NH} . Additionally, the Asymptotic Equipartition principle [4, p. 80] states that, for a sufficiently large distribution of i.i.d random variables, the outcome is almost certain to be one from the typical set.

This principle is similar to the weak law of large numbers which states that for a sampling, the sample mean converges in probability to the expected value of the distribution as the number of samples collected increases.

2.b) Shannon’s Source Coding Theorem

Shannon’s source coding theorem states that for N i.i.d random variables with entropy $H(X)$ can be compressed into more than $NH(X)$ bits with negligible risk of information loss, as $N \rightarrow \infty$. In other words, it is virtually certain that information will be lost if they are compressed to fewer than NH bits [4, p. 80].

3) Variable Length Codes

Variable length symbol codes are those that are mapped to one symbol at a time, unlike N -length strings. These are always lossless but also means that sometimes, they encode to slightly longer strings than the source. The key is to optimally assign shorter codes to more probable strings and longer ones to the more probable ones.

A binary symbol code [4, p. 92] for X maps for all elements of A_X to codes in the range $\{0, 1\}^+$. The codeword for any given symbol x is given by $c(x)$ and $l(x)$ denotes the length of the corresponding codeword. The extended code, given by C^+ , is a mapping from A^+ to $\{0, 1\}^+$, obtained by concatenation and without punctuation, of the corresponding codewords:

$$c^+(x_1x_2...x_N) = c(x_1)c(x_2)...c(x_N) \quad (5)$$

Such a code is uniquely decodeable, if under the extended code $C(X)^+$, no two distinct strings in A^+ have the same codeword. Furthermore, a **prefix code** is one where no codeword is a prefix of another codeword. Prefix codes are ideal because they remove the ambiguity during decoding; how would a decoder decide which source string a code should be mapped to as it reads the codeword one symbol at a time?

The expected length of a symbol code for an ensemble X is given by [4, p. 93] ,

$$L(C, X) = \sum (x \in X) P(x) l(x) \quad (6)$$

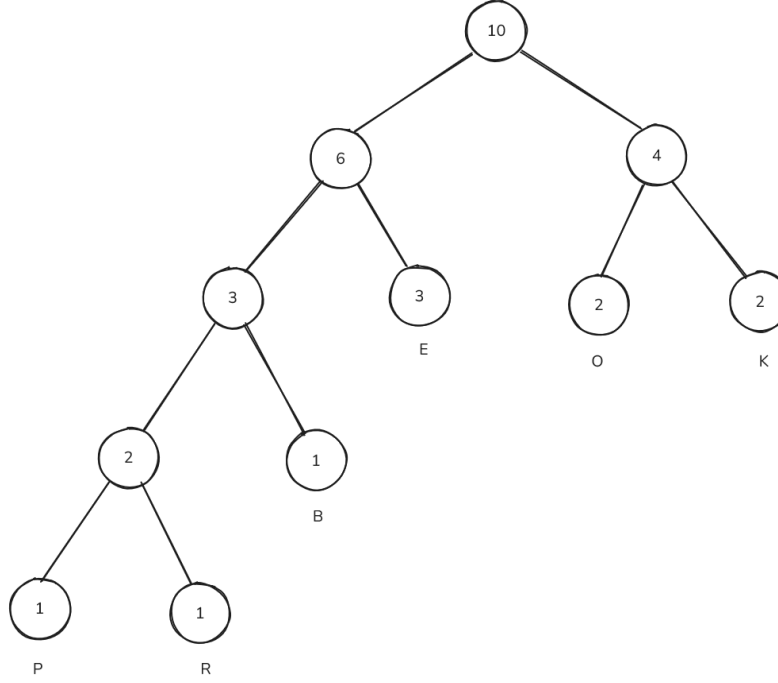
3.a) Huffman Codes

The goal is to minimize the expected length $L(C, X)$ of a symbol code. As it turns out, this is lower bounded by the entropy $H(X)$, i.e, we can't hope to compress a symbol code to lower than its entropy [4, p. 97].

Huffman coding aims to produce such an optimal prefix code, and it does so by a greedy approach and by building the binary tree in reverse, with the leaf elements first. [4, p. 99]

- 1 Take the two least probably elements from the alphabet. These two will be given the longest codewords, of equal length **and** they will only differ **in** the last digit.
- 2 Combine them into a single symbol **and repeat**.

Algorithm 1: Huffman Coding

Figure 1: A Huffman tree representation of the string **Bookkeeper**

a_i	p_i	$h(p_i)$	l_i	$c(a_i)$
B	0.1	3.32	3	001
o	0.2	2.32	2	10
k	0.2	2.32	2	11
e	0.3	1.73	2	01
p	0.1	3.32	4	0000
r	0.1	3.32	4	0001

Table 1: Code created by the Huffman algorithm

There are some caveats with Huffman codes. Firstly, it requires two passes through the source string: the first to calculate frequencies of each source symbol, and the second to build the Huffman tree and arrive at the codewords. Secondly, depending on the context, it can end up using more bits than is efficient for each symbol. Symbol frequencies can vary and Huffman coding does not handle a shifting distribution well.

3.a.i) Optimality

A Huffman code is an optimal symbol code. The proof provided in [4, p. 105] hinges on the fact that the algorithm chooses to give the two least probable symbols the same codeword length.

3.b) Dynamic Huffman Codes

In order to address the lack of flexibility of Huffman codes in the case of varying source distributions, dynamic or adaptive Huffman codes only require one-pass to encode the source string. Both the encoder and decoder have the same initial state and build the same Huffman tree and hence, the encoder does not need to communicate the structure of the tree to the decoder, leading to a shorter encoded string already.

There exist two algorithms for dynamic Huffman encoding:

- FGK, an implementation that was originally proposed independently by Faller, Gallager in [5], and improved by Knuth in [6],
- Vitter's algorithm proposed in [7] produces encodings that use less than one extra bit per letter.

Despite their differences relating to optimally swapping nodes in the tree, they share the same basic idea, with Vitter's algorithm being more optimal at the expense of being computationally slower. As they are one-pass algorithms that cannot pre-compute the distribution of the source symbols, they create dynamic codes that adapt to shifting probabilities of each symbol.

Given $M_t = a_1, a_2, \dots, a_t$ are the first t letters of the message so far, both the sender and the receiver modify their copy of the Huffman tree in the same way when the $(t + 1)$ st letter is sent, result in the Huffman tree for M_{t+1} . Such a Huffman tree of p leaves must fulfill the Sibling property or the Sibling rule [7, p. 6]:

- the p leaves have non-negative weights $w_1, w_2, \dots, w_p$, and the weight of each interior node is the sum of the weights of its children
- the nodes are numbered in a non-decreasing order by weight, such that nodes $2j - 1$ and $2j$ are siblings, for $1 \leq j \leq p - 1$ and their common parent is higher in the numbering

Consider a scenario with a source of alphabet size $t = 26$ and we have an incoming symbol $a_{i_{t+1}} = x$. The tree cannot be updated with the new symbol in the same way as it was done in Huffman's algorithm, as it could possibly break the Sibling property of the tree. The tree first must be rearranged such that updating the weight of the leaf node corresponding to the new symbol would still result in a valid Huffman tree.

The update can be done as follows:

1. Mark the leaf node of $a_{i_{t+1}}$ as the current node and swap it with the highest numbered node with the same weight.
2. Next mark the parent node of the latter as the new current node.
3. Repeat the process until we reach the root of the tree.
4. Increment the weight of corresponding leaf node for $a_{i_{t+1}}$ and also that of its ancestors.

The tree also contains a pseudo-node, sometimes known as the 0-node or the not-yet-traversed (NYT) node. This is used to represent any as yet unseen symbols from the source. When a new symbol is encountered, the 0-node is split to create two leaf nodes, one for the new symbol and the other becomes the new 0-node.

4) Stream Codes

From the above Huffman coding procedure, we are able to map binary code symbols efficiently and produce a tree representation that minimizes the expected codelength. However, its disadvantages still persist: they require multiple passes over the source, and they incur a small overhead over the ideal expected code length $H(X)$ of around 1 bit [4, p. 101].

4.a) Arithmetic Coding

Arithmetic coding alleviates some of these disadvantages. Rather than encoding per symbol like Huffman, arithmetic codes encode the entire source string as a binary fraction.

Using an interval $[0, 1)$, an arithmetic coding encoder divides this interval into sub-intervals, for each symbol that it reads from the string to be encoded, proportional to the probability of that symbol occurring next in the string. Any fractional value in the final interval can be chosen to represent the chosen message.

The binary fraction is a pointer to the sub-interval that results from narrowing the range $[0, 1)$ by using the probabilities of each symbol in the string.

```

1   $u := 0.0$ 
2   $v := 0.0$ 
3   $p := v - u$ 
4  for  $n = 1$  to  $N$  {
5      Compute the cumulative probabilities  $Q\_n$  and  $R\_n$ 
6       $v := u + pR_n(x_n \mid x_1, \dots, x_{n-1})$ 
7       $u := u + qR_n(x_n \mid x_1, \dots, x_{n-1})$ 
8       $p := v - u$ 
9  }
```

Algorithm 2: Arithmetic Coding

The lower and upper cumulative probabilities mentioned are defined as:

$$Q_n(a_i \mid x_1, \dots, x_{n-1}) \equiv \sum_{i'=1}^{i-1} P(x_n = a_{i'} \mid x_1, \dots, x_{n-1}) \quad (7)$$

$$R_n(a_i \mid x_1, \dots, x_{n-1}) \equiv \sum_{i'=1}^{i-1} P(x_n = a_{i'} \mid x_1, \dots, x_{n-1}) \quad (8)$$

A simpler analogy is considering a real line with the interval 0 to 1. This is the interval for the message before anything is transmitted. Each symbol processed by the encoder narrows down the interval needed to represent it. However, the interval reduces by a smaller degree when it encounters a more probable symbol as compared to that of a less probable one. This is how it ensures that the resulting binary fraction remains optimal [8]. However, despite their high optimality, arithmetic codes have historically been encumbered by patents and are often used in image and video compression formats such as JPEG.

4.b) Lempel-Ziv

The Lempel-Ziv family of algorithms and their derivatives are featured in a number of compression libraries such as zlib, compress, xz, etc. There are actually two variants of Lempel-Ziv that were proposed at around the same time, namely LZ77 and LZ78. They differ slightly in their implementation; LZ77 uses a sliding-window approach and LZ78 uses a dictionary to aid lookup, but their basic idea remains the same.

LZ compresses by replacing a substring occurring in the given source string by a reference/pointer to an earlier occurrence of that substring. It transmits such references as a reference-symbol pair, the

extra symbol being the extra 1 bit that differs from the substring, the latter acting as a prefix for the new symbol.

Therefore, each reference can be represented by $\lceil \log_2(r) \rceil$ bits, where r is the number of unique phrases in the source string. The dictionary is initialized with a null substring, so that wholly unique substrings can be transmitted with a reference-symbol pair by using a reference to the null string. Ideally, no bits would be required to convey the null string.

A decoder operates in the same manner as the coder and reconstructs the same dictionary as it passes through the transmitted values, replacing reference-symbol pairs by their actual symbol values as they occur.

Lempel-Ziv still has a few pitfalls; it will necessarily lengthen rarer substrings and without enough redundancy in the source string, the overhead of transmitting reference-symbol pairs to the decoder will negate any benefits of compression. Additionally, the extra bit used in the reference-pair can only take on two values, so the second time the same reference is seen, the decoder will know for certain what the extra bit value is supposed to be. Omitting this bit on the second occurrence can help reduce the length of the transmitted string.

However, for most purposes where there is a decent amount of redundancy in the source, LZ compresses well. Theoretically, the length per symbol of LZ encoding is asymptotically no greater than the entropy rate of the source. It should be able to compress a source without knowing its distribution and still achieve optimal compression nearly equal to the entropy rate. [9, p. 326]

5) DEFLATE Compression

DEFLATE [10] is a compression format most widely used in the gzip compression utility and the zlib compression library, as well as in libpng where it is used for compressing image data. It combines LZ77 sliding-window compression with either static or dynamic Huffman coding. The DEFLATE stream is a series of blocks, each block contains a Huffman tree and a portion of compressed data. Each Huffman tree is independent of each other, but the LZ77 compressed data can refer to strings found in prior blocks.

DEFLATE allows for multiple compression levels, indicated by a BFINAL byte in the header of each block:

- 00: no compression
- 01: compressed with fixed Huffman codes

- 02: compressed with dynamic Huffman codes
- 03: reserved

The LZ77 compressed data region in each block contains literal byte strings, i.e, input data sequences that have not been seen in the previous 32K bytes, and length-distance pairs which are pointers to duplicated strings. DEFLATE only looks for backwards distances within the previous 32K bytes, but they can be from a previous block of data. The length is represented by 8 bits.

DEFLATE reduces the length of bits required to represent the compressed data region by using a combined alphabet:

- literal values, in the case of uncompressed data blocks or strings not seen in the previous 32K bytes, as well as the lengths from length-distance pairs are drawn from the same alphabet. The values 0..255 represent the literal bytes, 256 indicates end-of-block, and values 257..285 along with some extra bits indicate length values of 3..258 bytes,
- backwards distance values are in the range 1..32768

While the details of DEFLATE's compression algorithm can vary according to different implementations, they roughly involve:

- LZ77 compression for the source continuously until the output buffer is full, at which point it designates it as the end of the current block, unless the block is designated to contain uncompressed data,
- compressing the LZ77 encoded data by Huffman encoding the literals and the length-distance pairs,
- the trees themselves are stored as a Huffman encoding of the code lengths [11].

The zlib implementation of DEFLATE decides what type of encoding to use for the compressed block by comparing the number of bytes occupied by the blocks by each method, i.e fixed Huffman and dynamic Huffman, and choosing the more efficient method [11]. It also provides multiple levels of compression which adjusts multiple parameters of deflate, including the sliding window size. Higher levels of compression improve the compression ratio at the expense of compression speed and memory usage.

Bibliography

- [1] Austin Whyte, “How Discord Reduced Websocket Traffic by 40%.” [Online]. Available: <https://discord.com/blog/how-discord-reduced-websocket-traffic-by-40-percent>
- [2] Facebook Open Source, “Zstandard - Fast real-time compression algorithm.” [Online]. Available: <https://github.com/facebook/zstd>
- [3] “LZ4 - Extremely fast compression.” [Online]. Available: <https://github.com/lz4/lz4>
- [4] D. J. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.
- [5] R. Gallager, “Variations on a theme by Huffman,” *IEEE Transactions on Information Theory*, vol. 24, no. 6, pp. 668–674, 1978, doi: 10.1109/TIT.1978.1055959.
- [6] D. E. Knuth, “Dynamic huffman coding,” *Journal of Algorithms*, vol. 6, no. 2, pp. 163–180, 1985, doi: [https://doi.org/10.1016/0196-6774\(85\)90036-7](https://doi.org/10.1016/0196-6774(85)90036-7).
- [7] J. S. Vitter, “Design and analysis of dynamic Huffman codes,” *J. ACM*, vol. 34, no. 4, pp. 825–845, Oct. 1987, doi: 10.1145/31846.42227.
- [8] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Commun. ACM*, vol. 30, no. 6, pp. 520–540, June 1987, doi: 10.1145/214762.214771.
- [9] T. M. Cover and J. A. Thomas, *Elements of Information Theory*. John Wiley & Sons, Ltd, 2005. doi: <https://doi.org/10.1002/047174882X>.
- [10] L. P. Deutsch, “DEFLATE Compressed Data Format Specification version 1.3.” [Online]. Available: <https://www.rfc-editor.org/info/rfc1951>
- [11] Mark Adler, “A massively spiffy yet delicately unobtrusive compression library.” [Online]. Available: <https://github.com/madler/zlib/>